4.3.4 remove

As is common with many data structures, the hardest operation is deletion. Once we have found the node to be deleted, we need to consider several possibilities.

If the node is a leaf, it can be deleted immediately. If the node has one child, the node can be deleted after its parent adjusts a link to bypass the node (we will draw the link directions explicitly for clarity). See Figure 4.24.

The complicated case deals with a node with two children. The general strategy is to replace the data of this node with the smallest data of the right subtree (which is easily found) and recursively delete that node (which is now empty). Because the smallest node in the right subtree cannot have a left child, the second `remove` is an easy one. Figure 4.25 shows an initial tree and the result of a deletion. The node to be deleted is the left child of the root; the key value is 2. It is replaced with the smallest data in its right subtree (3), and then that node is deleted as before.

The code in Figure 4.26 performs deletion. It is inefficient because it makes two passes down the tree to find and delete the smallest node in the right subtree when this is appropriate. It is easy to remove this inefficiency by writing a special `removeMin` method, and we have left it in only for simplicity.

If the number of deletions is expected to be small, then a popular strategy to use is **lazy deletion:** When an element is to be deleted, it is left in the tree and merely *marked* as being deleted. This is especially popular if duplicate items are present, because then the data member that keeps count of the frequency of appearance can be decremented. If the number of real nodes in the tree is the same as the number of "deleted" nodes, then the depth of the tree is only expected to go up by a small constant (why?), so there is a very small time penalty associated with lazy deletion. Also, if a deleted item is reinserted, the overhead of allocating a new cell is avoided.

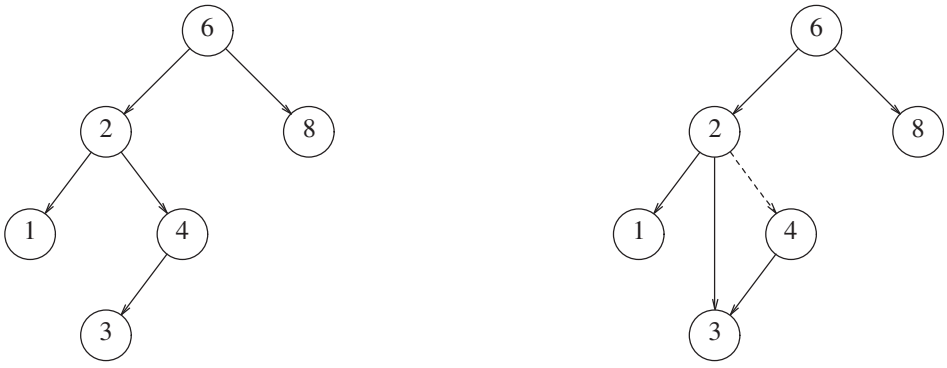

Figure 4.24 Deletion of a node (4) with one child, before and after

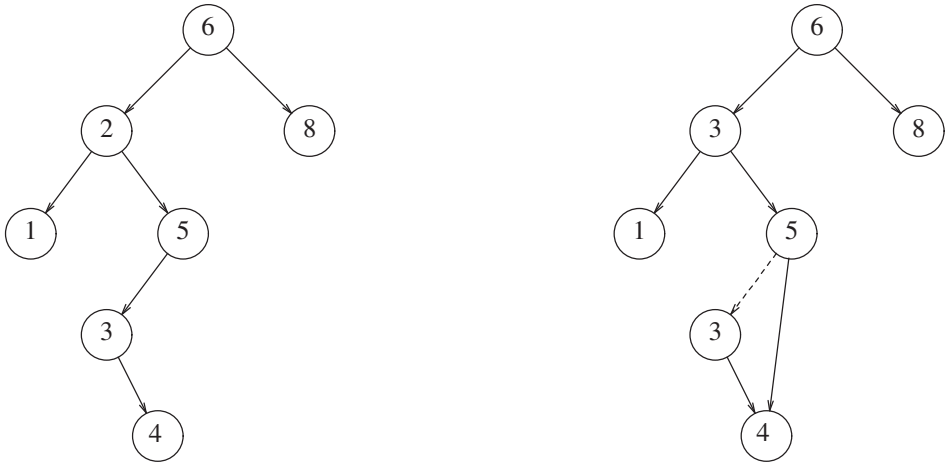

Figure 4.25 Deletion of a node (2) with two children, before and after

```
def delete(self, value):
    self.root = self._delete_recursive(self.root, value)

def _delete_recursive(self, current_node, value):
    if current_node is None:
        return current_node
    if value < current_node.value:
        current_node.left = self._delete_recursive(current_node.left, value)
    elif value > current_node.value:
        current_node.right = self._delete_recursive(current_node.right, value)
    else:
        if current_node.left is None:
            return current_node.right
        elif current_node.right is None:
            return current_node.left

        temp_node = self._min_value_node(current_node.right)
        current_node.value = temp_node.value
        current_node.right = self._delete_recursive(current_node.right, temp_node.value)


    return current_node
```

Figure 4.26 Deletion routine for binary search trees